AI WhatsApp Logistics System — Upgrade Walkthrough

What Changed

Complete refactor from an OpenAI-dependent system to a **fully local, free, rule-based** logistics platform testable via `POST /webhook`.

Architecture

```
graph TD
    A["curl / WhatsApp"] -->|POST /webhook| B["FastAPI"]
    B -->|dev mode: sync| C["Agent (State Machine)"]
    B -->|prod mode: async| D["Celery Worker"]
    D --> C
    C --> E["Mock AI Parser"]
    C --> F["Location Resolver"]
    C --> G["Redis State"]
    C --> H["PostgreSQL"]
    C --> I["Pricing Service"]

    J["Driver APIs"] -->|/accept /location /verify-weight| B
```

Files Modified/Created

File	Action	Purpose
`settings.py`	Modified	All API keys optional
`docker-compose.yml`	Modified	Plain Postgres, no Prometheus/Grafana
`requirements.txt`	Modified	Removed openai, sentry, geoalchemy2
`booking.py` (model)	Modified	+lat/lng, weight fraud, remote booking fields
`driver.py` (model)	Modified	Removed PostGIS dependency
`state_machine.py`	Rewritten	8 granular states, cancel, idempotency, tracking
`agent.py`	Rewritten	State-machine conversation handler, all 10 scenarios
`router.py`	Rewritten	Rule-based intent classifier
`mock_ai_parser.py`	**New**	Regex/keyword NLP parser
`location_resolver.py`	**New**	Known locations, coords parsing, Haversine
`webhook.py` (route)	Rewritten	Dual-format, synchronous dev mode
`driver.py` (route)	**New**	/accept, /location, /verify-weight
`pricing_service.py`	Rewritten	Haversine distance + weight-based pricing
`booking_service.py`	Rewritten	Full CRUD with new fields

`driver_service.py`	Rewritten	Nearest driver by Haversine
`tracking_service.py`	Rewritten	Maps links, Redis fallback
`main.py`	Simplified	No Sentry/Prometheus, mounts driver routes
`test_scenarios.sh`	**New**	All test cases as curl commands

All 10 Corrections Applied

#	Correction	Implementation
1	Redis idempotency	`state_machine.check_idempotency()` with SETNX + 24h TTL
2	State reset	"cancel"/"start over" keywords + auto-reset after booking
3	Independent location validation	Both pickup AND drop checked separately
4	Haversine distance	`location_resolver.haversine_distance()` for real km
5	Driver auto-assign	`find_nearest_available_driver()` sorts by distance
6	Tracking fallback	Returns "not available yet" when Redis empty
7	Remote booking validation	Cannot confirm without `passenger_phone`
8	Weight validation	Rejects negative, zero, non-numeric, >50000kg
9	Driver endpoint validation	Checks booking exists, driver exists, driver assigned
10	Standardized response	`{"reply": "...", "state": "...", "booking_id": ...}`

How to Test

1. Start the System

```
cd /Users/saidheerajreddypalle/Documents/AI_LOGISTICS
docker-compose up --build
```

2. Run All Tests

```
./test_scenarios.sh
```

3. Individual Test Commands

Ambiguous location:

```
curl -s -X POST http://localhost:8000/api/v1/webhook \
  -H "Content-Type: application/json" \
```

```
  -d '{"message_id":"t1","phone":"9999999999","text":"RGIA to Kondapur"}' | python3
-m json.tool
```

Full booking flow:

```
# Step 1: Start booking with exact coordinates
curl -s -X POST http://localhost:8000/api/v1/webhook \
  -H "Content-Type: application/json" \
  -d '{"message_id":"b1","phone":"8888888888","text":"Send truck from
17.2403,78.4294 to 17.4399,78.3489"}'

# Step 2: Provide weight
curl -s -X POST http://localhost:8000/api/v1/webhook \
  -H "Content-Type: application/json" \
  -d '{"message_id":"b2","phone":"8888888888","text":"200kg"}'

# Step 3: Confirm
curl -s -X POST http://localhost:8000/api/v1/webhook \
  -H "Content-Type: application/json" \
  -d '{"message_id":"b3","phone":"8888888888","text":"confirm"}'
```

Remote booking:

```
curl -s -X POST http://localhost:8000/api/v1/webhook \
  -H "Content-Type: application/json" \
  -d '{"message_id":"r1","phone":"7777777777","text":"Book for my friend"}'
```

Driver accept (use booking_id and driver_id from booking response):

```
curl -s -X POST http://localhost:8000/api/v1/driver/accept \
  -H "Content-Type: application/json" \
  -d '{"driver_id":"<UUID>","booking_id":"<UUID>"}'
```

Driver location update:

```
curl -s -X POST http://localhost:8000/api/v1/driver/location \
  -H "Content-Type: application/json" \
  -d '{"driver_id":"<UUID>","booking_id":"<UUID>","lat":17.385,"lng":78.487}'
```

Duplicate test:

```
# Send same message_id twice
curl -s -X POST http://localhost:8000/api/v1/webhook \
  -H "Content-Type: application/json" \
# Returns: {"state": "duplicate"}
```

Negative weight validation:

```
# First, start a booking to enter weight collection state
curl -s -X POST http://localhost:8000/api/v1/webhook \
  -H "Content-Type: application/json" \
  -d '{"message_id":"n1","phone":"555123","text":"Send from 17.2,78.4 to
17.4,78.5"}'

# Submit negative weight
curl -s -X POST http://localhost:8000/api/v1/webhook \
  -H "Content-Type: application/json" \
  -d '{"message_id":"n2","phone":"555123","text":"-150kg"}'
# Returns confirmation that weight must be positive
```

Troubleshooting

Database Connection Error (PostGIS Conflict)

If you previously ran the system with `postgis/postgis` and get a database connection refused error on startup, your old Docker volume needs to be deleted so the standard `postgres:15-alpine` image can initialize properly:

```
# Stop containers and delete the old volumes
docker-compose down -v
# Start fresh
docker-compose up --build
```